

DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING

Course Code: CS-324

Course Title: Machine Learning

Complex Engineering Problem

TE Batch 2023, Spring Semester 2026

Grading Rubric 1: Dataset Collection and Preprocessing (Milestone 2)

TERM PROJECT

Group Members:

Student No.	Name	Roll No.
S1	Muhammad Abdullah	23033
S2	Asra Siddiqui	23006
S3	Fatima Shahid	23007
S4		

CRITERIA AND SCALES				Marks Obtained			
				S1	S2	S3	S4
Criterion 1: Data collection, labeling, and dataset quality. (CPA-1, CPA-3) [4 marks]							
1	2	3	4				
Data collection or labeling is incomplete, incorrect, or poorly justified.	Data collection and labeling are partially completed with quality issues.	Data collection and labeling are complete and mostly correct.	Data collection and labeling are complete, accurate, and well justified, resulting in a high-quality dataset.				
Criterion 2: Preprocessing pipeline and exploratory data analysis (EDA). (CPA-1, CPA-2) [4 marks]							
1	2	3	4				
Preprocessing and EDA are missing or incorrectly applied.	Preprocessing or EDA is applied partially or superficially.	Preprocessing and EDA are applied correctly with minor gaps.	Preprocessing and EDA are comprehensive, correct, and well documented.				
Criterion 3: Report quality and adherence to format. [2 marks]							

1	2	3	4				
---	---	---	---	--	--	--	--

The report is poorly written and does not follow the given format.

The report partially follows the format and lacks clarity.

The report follows the format with minor formatting or clarity issues.

The report is clear, well written, and fully adheres to the given format and requirements.

Final Score = (Criteria_1_score) + (Criteria_2_score) + (Criteria_3_score) = _____

Teacher's Signature

Predicting Football League Standings Using Multi-Season Statistical Modeling

1- Introduction

This project aims to predict the final league positions of Premier League teams using machine learning. Match-level data from previous seasons is converted into team-level statistics such as goal difference and shots on target per match.

The model is trained on six seasons of historical data and predicts the rankings (positions 1–17) of teams in the following season. Only teams present in consecutive seasons are included to ensure consistency in the dataset.

The goal is to identify patterns in team performance and use them to forecast future league standings.

2. Data Collection Methods and Sources

The dataset for this project was constructed using real-world football match statistics collected from historical English Premier League seasons. The primary objective of the dataset is to enable prediction of team positions based on past performance.

Data was gathered for multiple seasons ranging from 2019–2020 to 2023–2024, ensuring sufficient temporal coverage to capture trends, consistency, and variability in team performance. The dataset includes match-level and season-level aggregated statistics.

The raw data was obtained from publicly available football datasets, which were then processed, cleaned, and transformed into structured formats suitable for machine learning tasks. This processed dataset served as the primary input for model training and evaluation.

2.1 Source Description

The dataset was compiled using the following sources:

[Football-Data.co.uk](https://www.football-data.co.uk)

Provided historical match data including goals, results, shots, and other match statistics. Data was available in CSV format for each season.

[Official Premier League](#)

Used as a reference to verify and correct inconsistencies in the raw dataset.
Helped ensure accuracy in team standings, points, and season summaries.

Constructed Dataset Structure

The data was transformed into structured season-wise tables: Each row represents a team's performance in a given season

Features include:

Goals For (GF)
Goals Against (GA)
Goal Difference (GD)
Wins, Losses, Draws
Shots and Shots on Target
Other performance metrics
Target Variable Design

The target variable is:

Points (PTS) of the next season
This allows the model to learn patterns from past performance and predict future outcomes.

2.2 Data Extraction

The dataset was processed and prepared using Python (Pandas).

Steps Involved:

Data Loading

CSV files for each season were loaded into dataframes.

Data Cleaning

Removed irrelevant columns
Handled missing or inconsistent values
Standardized column names across seasons

Feature Selection

Selected only relevant performance metrics for modeling
Ensured consistency across all seasons

Season-wise Table Creation

Converted match-level data into team-level aggregated statistics
Each season represented as a separate structured table

Target Mapping

Shifted the Points (PTS) column to the next season
Created input-output pairs for supervised learning

Final dataset contains:

Features: Current season stats

Target: Next season points

Final Dataset Characteristics

Covers multiple seasons (2019–2024)

Structured for supervised learning

Captures both offensive and defensive performance

Reflects real-world variability in team performance

3. Dataset statistics in Tabular form

3.1 Dataset Statistics

The dataset summary table provides an overview of the overall structure of the combined dataset used for model training and analysis. It includes key information such as the total number of records, number of features, and the distribution of numerical and categorical variables.

The dataset consists of 102 records aggregated from six seasons, representing teams that remained in the league across consecutive seasons. Most of the features are numerical, reflecting performance-based metrics such as goals scored, goals conceded, and goal difference

```
[86] summary_table = pd.DataFrame({
    "Attribute": [
        "Total Records",
        "Total Features",
        "Numerical Features",
        "Categorical Features",
        "Missing Values",
    ],
    "Value": [
        total_records,
        total_features,
        num_features,
        cat_features,
        missing_values,
    ]
})

summary_table
```

	Attribute	Value
0	Total Records	102
1	Total Features	33
2	Numerical Features	32
3	Categorical Features	1
4	Missing Values	0

3.2 Season-wise table structure:

Season 2015/16

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Liverpool	Team_Man_City	Team_Man_United	Team_Newcastle	Team_Southampton	Team_Tottenham	Team_West_Ham	Team_Wolves
0	20152019	30	32	2	4	95	23	72	0.84	81	...	0	0	1	0	0	0	0
1	20152019	30	30	7	1	80	22	67	0.95	99	...	0	1	0	0	0	0	0
2	20152019	30	21	9	8	63	39	24	0.21	66	...	0	0	0	0	0	0	0
3	20152019	30	23	2	13	67	39	28	4.97	99	...	0	0	0	0	0	1	0
4	20152019	30	21	7	10	73	51	22	4.47	86	...	0	0	0	0	0	0	0
5	20152019	30	19	9	10	66	54	11	0.92	66	...	0	0	1	0	0	0	0
6	20152019	30	16	9	13	47	46	1	3.97	59	...	0	0	0	0	0	0	1
7	20152019	30	15	9	14	54	48	8	4.45	49	...	0	0	0	0	0	0	0
8	20152019	30	15	7	10	51	48	3	4.84	62	...	1	0	0	0	0	0	0
9	20152019	30	15	7	10	52	55	-3	4.13	39	...	0	0	0	0	0	0	1
10	20152019	30	14	8	16	52	59	-7	3.97	34	...	0	0	0	0	0	0	1
11	20152019	30	14	7	17	51	53	-2	3.84	43	...	0	0	0	0	0	0	0
12	20152019	30	12	9	17	42	48	-6	3.74	44	...	0	0	0	1	0	0	0
13	20152019	30	13	6	19	56	70	-14	4.29	34	...	0	0	0	0	0	0	0
14	20152019	30	11	7	20	45	68	-23	3.03	54	...	0	0	0	0	0	0	0
15	20152019	30	9	12	17	45	65	-20	4.28	62	...	0	0	0	0	1	0	0
16	20152019	30	9	9	20	35	60	-25	2.84	41	...	0	0	0	0	0	0	0

17 rows x 27 columns

Season 2015/16

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Liverpool	Team_Man_City	Team_Man_United	Team_Newcastle	Team_Sheffield_United	Team_Southampton	Team_Tottenham	Team_West_Ham	Team_Wolves
0	20162020	30	32	3	3	85	33	52	0.98	89	...	0	1	0	0	0	0	0	0
1	20162020	30	29	3	9	102	35	67	0.92	88	...	0	0	1	0	0	0	0	0
2	20162020	30	18	12	8	66	39	30	0.96	74	...	0	0	0	1	0	0	0	0
3	20162020	30	20	6	12	69	54	15	0.97	67	...	0	0	0	0	0	0	0	0
4	20162020	30	18	8	12	67	41	26	0.90	66	...	1	0	0	0	0	0	0	0
5	20162020	30	16	11	11	61	47	14	4.21	62	...	0	0	0	0	0	0	1	0
6	20162020	30	15	14	9	51	40	11	3.82	45	...	0	0	0	0	0	0	0	1
7	20162020	30	14	14	10	66	48	8	3.97	61	...	0	0	0	0	0	0	0	0
8	20162020	30	14	12	12	39	39	0	3.03	23	...	0	0	0	0	1	0	0	0
9	20162020	30	15	9	14	43	50	-7	3.53	39	...	0	0	0	0	0	0	0	0
10	20162020	30	15	7	10	51	60	-9	4.76	43	...	0	0	0	0	0	1	0	0
11	20162020	30	13	10	15	44	66	-12	4.26	59	...	0	0	0	0	0	0	0	0
12	20162020	30	11	11	18	38	68	-26	3.45	45	...	0	0	0	0	1	0	0	0
13	20162020	30	11	10	17	31	60	-19	3.16	44	...	0	0	0	0	0	0	0	0
14	20162020	30	9	14	15	39	54	-15	3.74	41	...	0	0	0	0	0	0	0	0
15	20162020	30	10	9	19	49	62	-13	4.29	65	...	0	0	0	0	0	0	0	1
16	20162020	30	9	8	21	41	67	-26	3.96	55	...	0	0	0	0	0	0	0	0

17 rows x 27 columns

Season 2020/21

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Liverpool	Team_Man_City	Team_Man_United	Team_Newcastle	Team_Southampton	Team_Tottenham	Team_West_Ham	Team_Wolves	
0	20202021	30	27	5	0	83	32	51	0.79	63	...	0	0	0	1	0	0	0	0
1	20202021	30	21	11	0	73	44	29	0.58	68	...	0	0	0	0	1	0	0	0
2	20202021	30	20	9	0	68	42	26	0.83	62	...	0	0	1	0	0	0	0	0
3	20202021	30	19	10	0	58	36	22	0.50	74	...	0	0	0	0	0	0	0	0
4	20202021	30	20	0	12	68	56	18	4.87	62	...	0	1	0	0	0	0	0	0
5	20202021	30	19	8	11	62	47	15	4.34	66	...	0	0	0	0	0	0	0	1
6	20202021	30	18	8	12	68	45	23	4.83	71	...	0	0	0	0	0	0	1	0
7	20202021	30	18	7	13	55	39	16	4.00	69	...	0	0	0	0	0	0	0	0
8	20202021	30	18	5	15	62	54	8	5.24	38	...	1	0	0	0	0	0	0	0
9	20202021	30	17	8	13	47	48	-1	3.89	39	...	0	0	0	0	0	0	0	0
10	20202021	30	16	7	15	55	46	9	4.92	45	...	0	0	0	0	0	0	0	0
11	20202021	30	12	9	17	46	62	-16	3.74	49	...	0	0	0	0	0	1	0	0
12	20202021	30	12	9	17	46	62	-16	3.74	49	...	0	0	0	0	0	0	0	0

[illegible]

Season 2022/23

Season	Matches Played	W	D	L	GF	GA	GD	50%	PTS	net	Team Everton	Team Fulham	Team Liverpool	Team Man City	Team United	Team Newcastle	Team Mott's Forest	Team Tottenham	Team West Ham	Team Wolves
2022/2023	38	28	5	5	94	33	61	5.78	91	—	0	0	0	0	1	0	0	0	0	0
1	2022/2023	38	26	6	6	88	43	45	5.37	89	—	0	0	0	0	0	0	0	0	0
2	2022/2023	38	23	8	9	58	43	15	5.63	80	—	0	0	0	0	1	0	0	0	0
3	2022/2023	38	19	14	5	68	33	35	5.18	60	—	0	0	0	0	0	0	0	0	0
4	2022/2023	38	19	10	9	75	47	28	5.58	82	—	0	1	0	0	0	0	0	0	0
5	2022/2023	38	18	8	12	72	53	19	5.08	48	—	0	0	0	0	0	0	0	0	0
6	2022/2023	38	18	7	13	51	46	5	3.97	68	—	0	0	0	0	0	0	0	0	0
7	2022/2023	38	18	6	14	70	63	7	5.28	66	—	0	0	0	0	0	0	1	0	0
8	2022/2023	38	15	14	9	58	46	12	4.28	39	—	0	0	0	0	0	0	0	0	0
9	2022/2023	38	15	7	18	55	53	2	3.92	47	—	0	1	0	0	0	0	0	0	0
10	2022/2023	38	11	12	15	40	49	-9	3.58	49	—	0	0	0	0	0	0	0	0	0
11	2022/2023	38	11	11	16	38	47	-6	4.18	63	—	0	0	0	0	0	0	0	0	0
12	2022/2023	38	11	8	19	31	58	-27	3.29	45	—	0	0	0	0	0	0	0	0	1
13	2022/2023	38	11	7	20	42	55	-13	3.82	52	—	0	0	0	0	0	0	0	1	0
14	2022/2023	38	11	6	21	37	71	-34	3.50	48	—	0	0	0	0	0	0	0	0	0
15	2022/2023	38	9	11	18	38	68	-30	3.11	36	—	0	0	0	0	0	1	0	0	0
16	2022/2023	38	8	12	18	34	57	-23	4.00	48	—	1	0	0	0	0	0	0	0	0

17 rows x 27 columns

Season 2023/24

[illegible]

The target variable "PTS_next" demonstrates a wide range of values, indicating variation in team performance across seasons. Overall, the feature statistics confirm that the dataset captures meaningful differences between teams, making it suitable for predictive modeling.

```
feature_stats = all_tables_clean.describe().T[["mean", "std", "min", "max"]]
feature_stats
```

	mean	std	min	max
MatchesPlayed	38.000000	0.000000	38.00	38.00
W	16.313725	5.772897	8.00	32.00
D	8.500000	2.802933	2.00	15.00
L	13.186275	4.861916	1.00	21.00
GF	57.872549	17.221143	31.00	102.00
GA	51.068627	12.038108	22.00	79.00
GD	6.803922	26.052893	-37.00	73.00
SOT_per_match	4.633235	0.994302	2.84	7.32
PTS_next	56.343137	16.227037	23.00	99.00

4. Data Preprocessing Steps

Data preprocessing in the ML_CEP project was performed programmatically using Python (Pandas). The same pipeline was consistently applied across all seasons to ensure uniformity.

4.1 Data Cleaning

- Removed unnecessary columns and retained only relevant match statistics

```
season = "20192020"

home = pd.DataFrame({
    "Team": df19_20["HomeTeam"],
    "MatchesPlayed": 1,
    "W": (df19_20["FTR"] == "H").astype(int),
    "D": (df19_20["FTR"] == "D").astype(int),
    "L": (df19_20["FTR"] == "A").astype(int),
    "GF": df19_20["FTHG"],
    "GA": df19_20["FTAG"],
    "ShotsOnTarget": df19_20["HST"]
})

away = pd.DataFrame({
    "Team": df19_20["AwayTeam"],
    "MatchesPlayed": 1,
    "W": (df19_20["FTR"] == "A").astype(int),
    "D": (df19_20["FTR"] == "D").astype(int),
    "L": (df19_20["FTR"] == "H").astype(int),
    "GF": df19_20["FTAG"],
    "GA": df19_20["FTHG"],
    "ShotsOnTarget": df19_20["AST"]
})

teams = pd.concat([home, away])
```

- Ensured no duplicate match entries were present


```
table1920.duplicated().any()
np.False_
```

- Verified consistency of categorical values such as match results (H, D, A) [Reference : SnapShot 1]
- Checked for missing values.

[6] 0s

display(table1920.isnull().sum())

...	0
Season	0
MatchesPlayed	0
W	0
D	0
L	0
GF	0
GA	0
GD	0
SOT_per_match	0
PTS_next	0
Team_Arsenal	0
Team_Aston Villa	0
Team_Brighton	0
Team_Burnley	0
Team_Chelsea	0
Team_Crystal Palace	0
Team_Everton	0
Team_Leicester	0
Team_Liverpool	0
Team_Man City	0
Team_Man United	0
Team_Newcastle	0
Team_Sheffield United	0
Team_Southampton	0
Team_Tottenham	0
Team_West Ham	0
Team_Wolves	0

dtype: int64

4.2 Data Transformation (Match-Level to Team-Level)

- Split dataset into **Home** and **Away** team statistics
- Created separate DataFrames for home and away matches
- Standardized both formats to have same columns:
 - MatchesPlayed
 - Wins (W), Draws (D), Losses (L)
 - Goals For (GF), Goals Against (GA)
 - Shots on Target
- Combined both datasets using concatenation
- Aggregated team-wise statistics using groupby()

This step converted raw match-level data into structured team-level performance data.

```
season = "20192020"

home = pd.DataFrame({
    "Team": df19_20["HomeTeam"],
    "MatchesPlayed": 1,
    "W": (df19_20["FTR"] == "H").astype(int),
    "D": (df19_20["FTR"] == "D").astype(int),
    "L": (df19_20["FTR"] == "A").astype(int),
    "GF": df19_20["FTHG"],
    "GA": df19_20["FTAG"],
    "ShotsOnTarget": df19_20["HST"]
})

away = pd.DataFrame({
    "Team": df19_20["AwayTeam"],
    "MatchesPlayed": 1,
    "W": (df19_20["FTR"] == "A").astype(int),
    "D": (df19_20["FTR"] == "D").astype(int),
    "L": (df19_20["FTR"] == "H").astype(int),
    "GF": df19_20["FTAG"],
    "GA": df19_20["FTHG"],
    "ShotsOnTarget": df19_20["AST"]
})

teams = pd.concat([home, away])

table1920 = teams.groupby("Team").sum()
```

4.3 Feature Engineering

New meaningful features were created from the aggregated data:

- Goal Difference: $GD = GF - GA$

```
table1920["GD"] = table1920["GF"] - table1920["GA"]
```

- Points: $PTS = (W \times 3) + D$

```
table1920["PTS"] = table1920["W"]*3 + table1920["D"]
```

- Shots on Target per Match: $SOT_per_match = \text{Total Shots on Target} / \text{Matches Played}$

```
table1920["SOT_per_match"] = table1920["ShotsOnTarget"] / table1920["MatchesPlayed"]  
table1920 = table1920.drop(columns=["ShotsOnTarget"])
```

- League Position (POS): Assigned based on sorting by Points, Goal Difference, and Goals For.

```
table1920 = table1920.sort_values(["PTS", "GD", "GF"], ascending=False)  
table1920.insert(1, "POS", range(1, len(table1920)+1))
```

- Season Label: Added as a feature to distinguish different seasons

```
table1920["Season"] = season
```

4.4 Data Integration Across Seasons

- Created separate tables for consecutive seasons (e.g., 2019–2020 and 2020–2021)

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Leicester	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Sheffield United	Team_Southampton	Team_Tottenham	Team_West Ham	Team_Wolves
0 20192020	38	32	3	3	85	33	52	6.08	69	...	0	1	0	0	0	0	0	0	0	0

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Leeds	Team_Leicester	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Southampton	Team_Tottenham	Team_West Ham	Team_Wolves
0 20202021	38	27	5	6	83	32	51	5.76	93	...	0	0	0	1	0	0	0	0	0	0

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Leicester	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Southampton	Team_Tottenham	Team_Watford	Team_West Ham	Team_Wolves
0	20182019	38	32	2	4	95	23	72	6.84	81	...	0	0	1	0	0	0	0	0	0

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Everton	Team_Fulham	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Nott'm Forest	Team_Tottenham	Team_West Ham	Team_Wolves
0	20222024	38	28	7	3	96	34	62	7.32	71	...	0	0	0	1	0	0	0	0	0

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Leeds	Team_Leicester	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Southampton	Team_Tottenham	Team_West Ham	Team_Wolves
0	20212022	38	29	6	3	99	26	73	6.68	89	...	0	0	0	1	0	0	0	0	0

Season	MatchesPlayed	W	D	L	GF	GA	GD	SOT_per_match	PTS_next	...	Team_Everton	Team_Fulham	Team_Liverpool	Team_Man City	Team_Man United	Team_Newcastle	Team_Nott'm Forest	Team_Tottenham	Team_West Ham	Team_Wolves
0	20222023	38	28	5	5	94	33	61	5.76	91	...	0	0	0	1	0	0	0	0	0

- Computed **next season points (PTS_next)** as prediction target

```
table_20["PTS_next"] = table_20["W"]*3 + table_20["D"]
table_20 = table_20[["PTS_next"]].reset_index()
```

- Merged datasets using inner join on Team name

```
table1920 = table1920.merge(table_20, on="Team", how="inner")
```

- This ensured only teams present in both seasons (stable teams) were retained

4.5 Encoding

- Applied **One-Hot Encoding** on team names using `pd.get_dummies()`

```
team_dummies = pd.get_dummies(table1920["Team"], prefix="Team", dtype=int)
table1920 = table1920.drop(columns=["Team"])
```

This step made the dataset suitable for machine learning models.